

Análise Semântica



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Front-end

- Análise Léxica:
 - Quebra a entrada em palavras conhecidas como símbolos léxicos (tokens).
- Análise Sintática:
 - Analisa a estrutura de frases do programa.
- Análise Semântica:
 - Calcula o significado do programa.

Introdução

O parser define uma estrutura sintática ilustrada por uma árvore sintática, entretanto não há definição de significado:

- Qual o tipo da variável “x”?
- A variável “x” foi redeclarada?
- Existem identificadores que não foram declarados?
- Declarados, mas não usados?
- A expressão “ $x*y+d$ ” é consistente no tipo?
- Quantos argumentos tem a função `gamb()`?
- Os tipos dos argumentos da função `gamb()` são consistentes?

Tipos de dados

Dois propósitos principais:

- Fornecer um contexto implícito.
 - Pascal
 - $a+b$: real ou integer?
 - `New p`: alocação no heap de tamanho correto.
 - C++/Java:
 - `New m_type()`: chamada automática de construtor.

Tipos de dados

Dois propósitos principais:

- Limitar o conjunto de operações semanticamente válidas.
 - Registros não podem ser somados a caracteres, etc.
 - Tipos de parâmetros de funções.
 - Não conseguem prevenir qualquer tipo de erro.
 - Pegam um número de erros suficientes para ser de grande valor prático

Sistema de tipos

Podemos dizer que consiste de:

- Um mecanismo para definição de tipos e sua associação a certas construções da linguagem.
- Um conjunto de regras para:
 - Equivalência.
 - Compatibilidade.
 - Inferência.

Sistema de tipos

- Equivalência
 - Determinar quando os tipos de dois valores são os mesmos.
- Compatibilidade
 - Determinar quando um valor de um dado tipo pode ser usado em um certo contexto.
- Inferência
 - Determinar o tipo de uma expressão a partir dos tipos de seus componentes ou (as vezes) do contexto em volta.

Sistema de tipos

- Verificação de tipos
 - Assegura que o programa obedece as regras de compatibilidade da linguagem.
- Linguagem fortemente tipada (*strongly typed*)
 - são linguagem onde os objetos/variáveis tem um tipo bem definido e que precisa ser informado no momento de sua declaração. Nela todas as variáveis têm um tipo específico e seus tipos são importantes para a linguagem.

Sistema de tipos

- Linguagem estaticamente tipada (*statically typed*)
 - É fortemente tipada
 - Verificação de tipos feita em tempo de compilação.
 - Poucas linguagens satisfazem, se encarado estritamente.
 - Na prática, o termo é aplicado quando a maior parte da verificação é feita estaticamente.

Sistema de tipos

- Ada:
 - Fortemente tipada e em sua maioria estaticamente tipada.
- Pascal:
 - Maior parte estaticamente tipada.
 - Não é fortemente tipada: *untagged variant records*.

Sistema de tipos

- C:
 - Não é fortemente tipada.
 - Unions.
 - Subrotinas com número variável de parâmetros.
 - Interoperabilidade entre arrays e ponteiros.
 - Implementações de C não costumam checar qualquer coisa em tempo de execução.

Sistema de tipos

- Assembly:
 - Não tem sistema de tipos.
 - Operações de qualquer tipo podem ser aplicadas a valores em locais arbitrários (memória ou registradores).
- Verificação Dinâmica:
 - Operações verificam em tempo de execução se os tipos de seus operandos são aceitáveis.
 - Ex.: LIST, Smalltalk, Schem.

Sistema de tipos

- Verificação dinâmica:
 - Maior parte das verificações feita em tempo de execução.
 - Operações verificam em tempo de execução se os tipos de seus operandos são aceitáveis.
 - LISP, Smalltalk, Schem, Python, Perl,...
 - Podem ser mais flexíveis e sofisticados por terem informação de tempo de execução.
 - Só asseguram que uma dada execução do programa está correta.
 - É repetida para todas as execuções.

Analizador semântico

- Checa os tipos de cada expressão.
- Relaciona declarações de variáveis com seus usos.
- É caracterizado pela manipulação de tabelas de símbolos.
 - Mapeiam identificadores e seus tipos e localização.
- Declarações geram inclusões nas tabelas de símbolos.

Analizador semântico

- Uso geram consultas nas tabelas.
- Escopo
 - Determina a visibilidade.
 - Ex.: variáveis locais de um método são visíveis apenas dentro do método.
- Tabelas são também chamadas de ambientes.
- Ambientes são conjuntos de “amarrações” do tipo:
 - $\{g \mapsto \text{string}, a \mapsto \text{int}\}$

Analizador Semântico

```
1 class C {  
2   int a; int b; int c;  
3   public void m(){  
4     System.out.println(a+c);  
5     int j = a+b;  
6     String a = "hello";  
7     System.out.println(a);  
8     System.out.println(j);  
9     System.out.println(b);  
10  }  
11 }
```

- σ_0
- $\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$ (2)
- $\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$ (5)
- $\sigma_3 = \sigma_2 + \{a \mapsto \text{String}\}$ (6)
 - Qual o tipo de a após esta adição?
 - Para ambientes, $X+Y$ é diferente de $Y+X$.
- Diga em qual ambiente é realizada a consulta para cada uso das variáveis do programa.

Analizador Semântico

```
structure M = struct
  structure E = struct
    val a = 5;
  end
  structure N = struct
    val b = 10
    val a = E.a + b
  end
  structure D = struct
    val d = E.a + N.a
  end
end
```

(a) An example in ML

```
package M;
class E {
  static int a = 5;
}
class N {
  static int b = 10;
  static int a = E.a + b;
}
class D {
  static int d = E.a + N.a;
}
```

(b) An example in Java

Como implementar a tabela de símbolos?

- Existem duas escolhas para se implementar a tabela de símbolos:
 - Usando um estilo funcional: mantemos o estado σ_1 enquanto criamos os estados σ_2 e σ_3 .
 - Assim, quando precisarmos de σ_1 novamente ele estará pronto para ser usado.
 - Usando o estilo imperativo: no estilo imperativo, modificamos σ_1 até ele se torna σ_2 .
 - Essa atualização destrutiva “destrói” σ_1 ; enquanto σ_2 existe, não podemos consultar a tabela σ_1 .
 - Mas quando terminamos com σ_2 , podemos “desfazer” até termos novamente σ_1 .

Implementação eficiente da tabela de símbolos (Imperativa)

- Como um programa grande pode conter milhares de identificadores distintos, a tabela de símbolos deve permitir uma busca eficiente.
- A implementação do ambiente usando o estilo imperativo, normalmente, usa tabelas *hash*, que são muito eficientes.
- A operação $\sigma' = \sigma + \{a \mapsto \tau\}$ é implementada inserindo τ na tabela *hash* com a chave a .
- Uma simples tabela *hash* com encadeamento externo funciona bem e permite fácil remoção.
- Vamos precisar deletar $\{a \mapsto \tau\}$ para recuperar σ no fim do escopo de a .

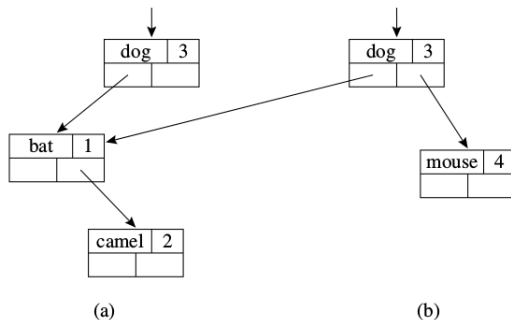
Implementação eficiente da tabela de símbolos (Funcional)

- No estilo funcional, queremos computar $\sigma' = \sigma + \{a \mapsto \tau\}$ de modo que continuamos com σ disponível.
- Desse modo, em vez de “alterar” a tabela adicionando uma amarração, criamos uma nova tabela computando a “soma” de uma tabela existente e uma nova amarração.
- Por exemplo, quando fazemos a adição $7 + 8$ não alteramos o valor de 7 pela adição de 8; nós criamos um novo valor, 15 - e o 7 continuando disponível para outras computações.

Implementação eficiente da tabela de símbolos (Funcional)

- Entretanto, atualizações não destrutivas não são eficientes para tabelas *hash*.
- Uma forma de implementar tal estilo eficientemente é usando árvores de busca binária.
- Considere, por exemplo, uma árvore de busca que representa o mapeamento.
 $m_1 = \{\text{bat} \mapsto 1, \text{camel} \mapsto 2, \text{dog} \mapsto 3\}$
- Podemos adicionar a amarração $\text{mouse} \mapsto 4$, criando o mapeamento m_2 sem destruir o mapeamento m_1 .

Implementação eficiente da tabela de símbolos (Funcional)



Análise Semântica



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)